

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS-553)

B.Tech CSE 5th SEMESTER

SUBMMITED TO:

MR. RASHAN RAJ

SUBMMITED BY:

MANKIRAT SINGH

ID: BCS2024542

ROLL NO: 2410303257

Lab Assignment 3: Bubble Sort

Experiment No. 3.1 - One pass of bubble sort

Experiment Title:

One pass of bubble sort

Aim:

To perform a single pass of bubble sort on an array.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Bubble sort repeatedly compares two neighbouring elements and swaps them if the left one is larger. After each pass the largest remaining value moves to its final position at the end, like a bubble rising to the surface. The sort needs an outer loop for the passes and an inner loop for the comparisons. Its worst-case time is $O(n^2)$.

Algorithm / Procedure:

1. Create the array {5, 1, 4, 2, 8}.
2. For j = 0 to length - 2 compare a[j] and a[j + 1].
3. If a[j] > a[j + 1], swap them using a temporary variable.
4. Print every value of the array.

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 3, Task 1
// Question: Run one pass of bubble sort on {5, 1, 4, 2, 8}. In the
// pass, compare each value with the next and swap when the left one
// is larger. Print each value of the array on its own line.

public class Main {
    public static void main(String[] args) {
        int[] a = {5, 1, 4, 2, 8};

        for (int j = 0; j < a.length - 1; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
```

```
        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

Input:

Fixed array declared in the program: {5, 1, 4, 2, 8}.

Output:

```
1
4
2
5
8
```

Time / Space Complexity: Time O(n); Space O(1).

Result:

The program was compiled and executed successfully. One pass moved the value 5 towards the end; the array after one pass is 1, 4, 2, 5, 8.

Viva-Voce Questions:

- What happens to the largest value after one pass?
- Why is a temporary variable needed for a swap?
- How many comparisons occur in one pass of n elements?

Experiment No. 3.2 - Bubble sort of {5, 1, 4, 2, 8}

Experiment Title:

Bubble sort of {5, 1, 4, 2, 8}

Aim:

To sort an array completely using bubble sort with an outer and an inner loop.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Bubble sort repeatedly compares two neighbouring elements and swaps them if the left one is larger. After each pass the largest remaining value moves to its final position at the end, like a bubble rising to the surface. The sort needs an outer loop for the passes and an inner loop for the comparisons. Its worst-case time is $O(n^2)$.

Algorithm / Procedure:

1. Create the array {5, 1, 4, 2, 8}.
2. Outer loop i runs for the passes (0 to length - 2).
3. Inner loop j compares a[j] and a[j + 1] up to length - 1 - i and swaps if out of order.
4. Print every value of the sorted array.

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 3, Task 2
// Question: Sort the array {5, 1, 4, 2, 8} with bubble sort, using
// one outer loop for the passes and one inner loop for the
// comparisons. Print each value of the sorted array on its own line.

public class Main {
    public static void main(String[] args) {
        int[] a = {5, 1, 4, 2, 8};

        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                }
            }
        }

        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
```

```
}
```

Input:

Fixed array declared in the program: {5, 1, 4, 2, 8}.

Output:

```
1
2
4
5
8
```

Time / Space Complexity: Time $O(n^2)$ worst case; Space O(1).

Result:

The program was compiled and executed successfully. The array was sorted in ascending order: 1, 2, 4, 5, 8.

Viva-Voce Questions:

- What is the worst-case time complexity of bubble sort?
- Why does the inner loop run only up to length - 1 - i?
- Is bubble sort stable?

Experiment No. 3.3 - Bubble sort of {9, 7, 3}

Experiment Title:

Bubble sort of {9, 7, 3}

Aim:

To sort a small array in reverse order using bubble sort.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Bubble sort repeatedly compares two neighbouring elements and swaps them if the left one is larger. After each pass the largest remaining value moves to its final position at the end, like a bubble rising to the surface. The sort needs an outer loop for the passes and an inner loop for the comparisons. Its worst-case time is $O(n^2)$.

Algorithm / Procedure:

1. Create the array {9, 7, 3}.
2. Run the outer loop for the passes and the inner loop for the comparisons.
3. Swap neighbours when the left one is larger.
4. Print every value of the sorted array.

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 3, Task 3
// Question: Sort the array {9, 7, 3} with bubble sort and print each
// value of the sorted array on its own line.

public class Main {
    public static void main(String[] args) {
        int[] a = {9, 7, 3};

        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                }
            }
        }

        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

Input:

Fixed array declared in the program: {9, 7, 3}.

Output:

```
3
7
9
```

Time / Space Complexity: Time $O(n^2)$; Space O(1).

Result:

The program was compiled and executed successfully. The array {9, 7, 3} was sorted to 3, 7, 9.

Viva-Voce Questions:

- Which input is the worst case for bubble sort?
- How many swaps are needed for {9, 7, 3}?
- How can bubble sort be stopped early on a sorted array?